

Programmeringsparadigm 97b/plan/f23/

Ändrad
11 Dec
11:01

Föreläsning 23

Tutti frutti III

C-programmeraren älskar att kunna fatta sig kort.

Nyckelorden och funktionsnamnen är korta.

Men det är inte bara fråga om lättja, det finns rationella skäl till dessa egenheter hos språket. Det handlar om att programmen ska arbeta effektivt.

C-programmet översätts av en kompilator till assembler. En del av de kortfattade uttrycksmöjligheterna handlar om att kompilatorn ska kunna generera effektiv kod.

Exempel, i C kan man skriva:

```
do {
    ...
} while((ch = getchar()) != EOF)
```

Medan man i andra programspråk måste skriva:

```
do {
    ...
    ch = getchar()
} while(ch != EOF)
```

I den första versionen kan kompilatorn enklare se att `ch` kan få ligga kvar i ackumulator-registret:

```
Dol: ....
    LoadWith    getchar()
    StoreIn      &ch
    LoadFrom    &ch /* Onödigt, ty ch är redan i Acc */
    CompareTo    EOF
    JumpIfNotZero Dol
```

Observera dock att detta *inte* är en garanti för att kompilatorn faktiskt stryker den onödiga raden!

Kommaoperatoren

I stället för

```
j = 10;
for (i = 0; i <= 10; i++){
    printf("%d %d", i, j);
    j++;
}
```

kan man skriva

```
for (i = 0, j = 10; i <= 10; i++, j++)
    printf("%d %d", i, j);
```

Kommaoperatoren gör i stort sett ingenting. Se den bara som ett nytt sätt att separera satser från varandra -- istället för att använda semikolon. Skillnaden är att kommatecknet fungerar på fler ställen.

Ett kommauttryck har ett värde, nämligen värdet av det första uttrycket. Resten av värdena spelar ingen roll för hela uttryckets värde.

Därför kan man skriva de mest bizarra saker, som till exempel:

```
i = 2, z++, printf("Oj");
```

för att sätta `i` till 2. Det händer mycket därefter, men `i` blir bara 2.

I `for`-satsen ovan var vi bara intresserade av att klämma in lite extra instruktioner i initierings- och uppdateringsdelarna av `for`-loopen. Vi var inte intresserade av kommauttryckens värden.

Jag har inget bra exempel där denna möjlighet utnyttjas (att ta om hand uttryckets värde). `For`-loopen är det enda användandet jag själv stött på, efter vad jag kan minnas.

Flerdimensionella variabler

Inget hindrar oss från att deklarera vektorer vars element är vektorer. I följande kod deklarerar jag 30 stycken strängar, var och en på 40 tecken vardera.

```
char crossword[30][40];

strcpy(crossword[12], "Hejsan");
crossword[16][20] = crossword[12][3]; /* = 's' */
```

Man säger att `crossword` är en *tvådimensionell* array. Det är tillåtet att deklarera arrayer av ännu högre dimensioner. Det är bara att haka på fler index.

Skillnaden mellan arrayer och pekare

Ni känner mig, jag ljugar ofta. Men det är *pia fraus* -- för att göra det enklare för er. Nu ska jag ta tillbaka en gammal lögn innan jag glömmer bort den.

Jag har sagt att arraynamn är pekare. Men pekare och arraynamn är inte riktigt samma sak -- däremot kan de användas som om de var samma sak, nästan jämnt. Det finns bara ett undantag.

Följande deklaration får demonstrera skillnaden:

```
double *p, x[3];
```

Ovanstående reserverar plats för fyra variabler. Inte fem som jag sagt tidigare. Variablerna heter `p`, `x[0]`, `x[1]` och `x[2]`. Det skapas *inte* plats för en pekarvariabel »`x`« som innehåller adressen till `x[0]` -- `&x[0]`.

Men om vi i programmet skriver `x`, byter kompilatorn ut

det mot `&x[0]`. Därför kan vi använda `nr` som om den var en pekare. Det enda vi inte kan göra är att tilldela `x` ett värde.

När vi tar emot en array som parameter är det dock alltid som en *pekare* den tas emot. Trots att vi oftast skriver exempelvis:

```
int strcmp(char u[], char v[]);
```

Men detta är bara en notation för att påminna läsaren om att `u` och `v` pekar ut arrayer. De flesta C-programmerare skriver:

```
int strcmp(char *u, char *v);
```

typedef

Nyckelordet `typedef` kan användas till mer än att definiera `struct`ar. Vilka typer som helst kan namnges.

■ Exempel -- döpa om enkla typer:

```
typedef int heltal;
typedef double flyttal;
```

Nu kan vi deklarera variabler av dessa typer:

```
heltal i, j;
flyttal x, y[20], *ptr;
```

Ovanstående deklarationer är hel likvärdiga med att vi deklarerat variablerna som `int` respektive `double`.

Så vad ska det då tjäna till? undrar du kanske. Varför »döpa om« `int` och `double` -- de heter väl bra som de redan heter?

Svar: jämför det med att definiera makron! När vi ändrar definitionen av ett makro, ändrar vi på alla ställen där makrot används -- det kan vara en kraftigt arbetsbesparande åtgärd.

Om du vill ändra typer i ditt program -- om du kanske kommer på att en `int` inte räcker till -- att du behöver en `unsigned long int` istället. Då behöver du bara ändra i typdefinitionen, så följer denna ändring med i hela programmet.

Istället för att behöva ändra på tretton ställen från `int` till `unsigned long int`, behöver du bara ändra i typdefinitionen.

■ Exempel: i filen `sys/types` finns några typdefinitioner. Bland annat definieras typen `time_t` som används för att lagra antalet sekunder som förflutit sedan 1 januari år 1970.

Typen `time_t` är en `long`. I alla fall på mitt system. Just nu. Det fina i kråksången är att om `time_t` skulle omdefinieras -- kanske till en `unsigned long` -- så behöver

jag bara kompilera om mitt program. Och allt fungerar igen.

Smart, eller hur?

■ Exempel: namnge sammansatta typer. Syntaxen är lite märklig, men tänk såhär: deklarationen av typen liknar hur variabeln annars skulle ha deklarerats.

```
typedef double vector[3];
typedef char string[100];
typedef struct {
    double re;
    double im;
} complex;
typedef int *intpointer;
```

Ovanstående definitioner gör att vi kan definiera följande variabler.

```
vector x, y; /* dvs: double x[3], y[3] */
string s1, s2; /* dvs: char s1[100], s2[100] */
complex A, B;
intpointer ip; /* dvs: int *ip */
int i;
```

Som vi sedan kan använda:

```
x[0] = y[1];
strcpy(s1, "Hejsan");
printf("%s%s", s1, s2);
A.re = y[1];
A.im = y[2];
B = A;
ip = &i;
```

static

Funktionen `evaluate()` på [förra föreläsningen](#) har parametrarna `line` och `x`, och variablerna `y`, `z`, `op` och `function`.

Alla dessa variabler och parametrar skapas varje gång funktionen anropas, och förstörs när funktionen exekverat färdigt. Det betyder att vi inte kan räkna med att till exempel `op` har kvar sitt gamla värde när funktionen anropas nästa gång.

»Skapas« och »förstörs« är möjligen något för kraftiga metaforer.

Det enda som egentligen händer är att datorn bestämmer var variablerna och parametrarna tillfälligt skall lagras. När funktionen exekverat färdigt glömmar datorn var han lade dem.

Ibland vill vi dock att variabler ska behålla sina värden mellan anropen. Då stoppar vi in nyckelordet `static` framför variabeldeklarationen.

■ Jag har skrivit en funktion `timer()` som kan användas som tidtagare. Ett anrop returnerar antalet sekunder som

gått sedan det förra anropet.

Jag utnyttjar funktionen `time()` som returnerar antalet sekunder som passerat sedan 1 januari år 1970.

```
8  time_t timer()
9  {
10     static time_t  lastTime, thisTime, timePassed;
11
12     time(&thisTime);
13     timePassed = thisTime - lastTime;
14     lastTime = thisTime;
15
16     return timePassed;
17 }
18
19
20 void main()
21 {
22     int ch;
23
24     timer();
25     while(1){
26         ch = fgetc(stdin);
27         printf("Time passed = %ld seconds\n", timer());
28     }
29 }
```

Variabeln `lastTime` är deklarerad `static` -- det betyder att den behåller sitt värde mellan anropen. Därför kan jag subtrahera från det nya värdet `thisTime` för att få fram tiden mellan anropen.

I huvudprogrammet anropar jag `timer()` en gång för att ge `lastTime` ett startvärde. Därefter mäter `timer()` tiden mellan mina tryckningar på returtangenten.

Nästa lektion

